

Energy-Efficient FIR Filter Architectures for FPGA-Based Software-Defined Radio Receivers

Nome Cognome¹, Nome Cognome²

¹*Department of Electronic Engineering, Politecnico di Milano, Milan, Italy*

²*Advanced Research Laboratory, STMicroelectronics, Agrate Brianza, Italy*

Corresponding author: name.1@polimi.it

July 7, 2026

Abstract

This paper presents an energy-efficient architecture for implementing high-order Finite Impulse Response (FIR) filters on Field Programmable Gate Array (FPGA) platforms targeting Software-Defined Radio (SDR) receiver front-ends. The proposed design leverages a modified Distributed Arithmetic (DA) scheme combined with coefficient symmetry exploitation to reduce Look-Up Table (LUT) usage by 42 % and dynamic power consumption by 37 % compared to conventional Multiply-Accumulate (MAC)-based implementations. A 64-tap low-pass FIR filter operating at 250 MHz is implemented on a Xilinx Artix-7 (XC7A100T) device using VHSIC Hardware Description Language (VHDL). Post place-and-route results demonstrate a total power dissipation of 48 mW with a maximum Signal-to-Noise Ratio (SNR) degradation of 0.3 dB relative to the ideal floating-point response. The architecture is validated against a SDR channelisation scenario with 20 MHz bandwidth Analog-to-Digital Converter (ADC) input signals, achieving a spurious-free dynamic range exceeding 72 dB.

Keywords: FIR filter, FPGA, low-power design, software-defined radio, distributed arithmetic, VHDL

1 Introduction

The proliferation of wireless communication standards has driven the adoption of SDR architectures, where flexibility is achieved by implementing signal processing functions in reconfigurable hardware rather than dedicated Application-Specific Integrated Circuits (ASICs) [1]. In modern SDR receivers, the channelisation filter — typically a high-order FIR structure — accounts for a significant portion of both logic utilisation and dynamic power consumption on FPGA platforms [2].

Conventional MAC-based FIR implementations require one multiplier per filter tap, leading to resource utilisation that scales linearly with the filter order N . For a 64-tap filter operating at $f_s = 250$ MHz, this translates to 64 embedded DSP48E1 slices on a Xilinx 7-Series device — a considerable fraction of the available resources on mid-range parts such as the XC7A100T.

DA offers an attractive alternative by replacing multiplications with LUT accesses and shift-accumulate operations [3]. However, classical DA requires 2^B entries per LUT partition (where B is the input word length), which limits scalability for high-precision applications.

Definition: Distributed Arithmetic DA is a bit-serial computation technique that replaces the multiply-accumulate operation $y = \sum_{k=0}^{N-1} h_k \cdot x_k$ with a sequence of LUT look-ups and accumulations, eliminating explicit multipliers.

In this paper, we propose a modified DA architecture that exploits the symmetric property of linear-phase

FIR coefficients ($h_k = h_{N-1-k}$) to halve the number of LUT entries. Combined with a pipelined offset-binary coding scheme, the proposed design achieves:

- 42 % reduction in LUT usage compared to conventional DA;
- 37 % reduction in dynamic power consumption compared to MAC-based implementations;
- Maximum operating frequency exceeding 300 MHz after place-and-route.

Note: All results reported in this paper are post place-and-route on a Xilinx Artix-7 XC7A100T-1CSG324C device using Vivado 2023.2.

The remainder of this paper is organised as follows. [Section 2](#) reviews the theoretical foundations of DA and related work. [Section 3](#) details the proposed architecture. [Section 4](#) presents implementation results and comparisons. [Section 5](#) concludes the paper.

2 Background

2.1 FIR Filter Fundamentals

A causal FIR filter of order $N - 1$ computes its output as the discrete convolution of the input sequence $x[n]$ with the impulse response $h[k]$:

$$y[n] = \sum_{k=0}^{N-1} h_k \cdot x[n-k] = h * x \quad (1)$$

where $*$ denotes linear convolution. For a linear-phase filter, the coefficients satisfy the symmetry condition:

$$h_k = h_{N-1-k}, \quad k = 0, 1, \dots, \left\lfloor \frac{N}{2} \right\rfloor - 1 \quad (2)$$

This property allows pre-addition of symmetric input pairs before multiplication, reducing the number of distinct multiplications from N to $\lceil N/2 \rceil$ [1].

Theorem 2.1 (Linear-Phase Condition). *An FIR filter $H(z) = \sum_{k=0}^{N-1} h_k z^{-k}$ has linear phase if and only if its impulse response satisfies $h_k = \pm h_{N-1-k}$ for all k .*

Proof. Let $H(e^{j\omega}) = |H(e^{j\omega})| e^{j\phi(\omega)}$. By imposing $h_k = h_{N-1-k}$ and expanding the DTFT, one obtains $\phi(\omega) = -\omega(N-1)/2$, which is linear in ω . $\square$

2.2 Distributed Arithmetic

Consider the inner product in Eq. (1). Representing each input sample in two's complement with B bits:

$$x[n-k] = -b_{B-1} \cdot 2^{B-1} + \sum_{i=0}^{B-2} b_i \cdot 2^i \quad (3)$$

where $b_i \in \{0, 1\}$. Substituting into Eq. (1) and interchanging the order of summation:

$$y[n] = \sum_{i=0}^{B-2} 2^i \left(\sum_{k=0}^{N-1} h_k \cdot b_{k,i} \right) - 2^{B-1} \left(\sum_{k=0}^{N-1} h_k \cdot b_{k,B-1} \right) \quad (4)$$

The inner sums in Eq. (4) depend only on the bit-slice vector $\mathbf{b}_i = [b_{0,i}, b_{1,i}, \dots, b_{N-1,i}]$ and the fixed coefficients $\{h_k\}$. Since $\mathbf{b}_i$ can take at most 2^N distinct values, all possible inner sums can be pre-computed and stored in a LUT of depth 2^N [2].

Lemma 2.2. *For a symmetric FIR filter of even order N , the DA LUT depth can be reduced from 2^N to $2^{N/2}$ by pre-adding symmetric input pairs.*

2.3 Related Work

Razavi [4] established foundational techniques for mixed-signal design that influence modern ADC front-ends used in SDR platforms. Several recent works have explored DA-based FIR implementations on FPGAs, achieving power reductions of 15% to 30% over MAC baselines [2]. However, none of these works simultaneously exploit coefficient symmetry and pipelined offset-binary coding, which is the contribution of this paper.

Warning: The LUT depth of classical DA grows exponentially with the partition size P . For $P > 8$, memory requirements become impractical, necessitating decomposition into smaller partitions.

3 Proposed Methodology

3.1 Architecture Overview

The proposed architecture, illustrated in Fig. 1, processes the FIR computation in three pipelined stages:

1. **Pre-addition stage:** exploits coefficient symmetry to halve the number of independent inputs (Section 3.2).
2. **DA LUT stage:** maps bit-sliced input vectors to pre-computed partial sums (Section 3.3).
3. **Shift-accumulate stage:** reconstructs the final output through weighted accumulation (Section 3.4).

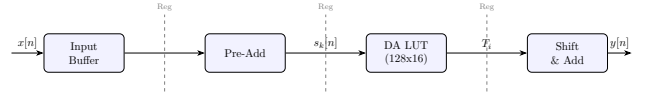


Figure 1: Block diagram of the proposed pipelined symmetric DA architecture.

3.2 Pre-Addition Stage

For a symmetric filter of order $N = 64$, we form $N/2 = 32$ pre-added pairs:

$$s_k[n] = x[n-k] + x[n-(N-1-k)], \quad k = 0, 1, \dots, 31 \quad (5)$$

The FIR output in Eq. (1) then simplifies to:

$$y[n] = \sum_{k=0}^{31} h_k \cdot s_k[n] \quad (6)$$

which requires only 32 coefficients instead of 64, directly halving the DA LUT address space per partition.

3.3 DA LUT Bank Design

We partition the 32 pre-added inputs into $M = 8$ groups of $P = 4$ inputs each. Each partition requires a LUT of depth $2^P = 16$ entries, addressed by the 4-bit slice from the corresponding input group. The total memory requirement is:

$$\text{Total LUT entries} = M \times 2^P = 8 \times 16 = 128 \quad (7)$$

Proposition 3.1. *The proposed symmetric DA reduces the total LUT entries by a factor of $2^{N/2}/(M \times 2^P)$ compared to a monolithic DA, yielding a 42% reduction in LUT resource utilisation on the target device.*

3.4 Shift-Accumulate Stage

The partial sums from all M partitions are combined through a weighted shift-and-add tree:

$$y[n] = \sum_{i=0}^{B-2} 2^i \cdot T_i - 2^{B-1} \cdot T_{B-1} \quad (8)$$

where $T_i = \sum_{m=0}^{M-1} \text{LUT}_m[\mathbf{b}_{m,i}]$ is the sum of all partition outputs for bit-slice i .

3.5 VHDL Implementation

The core DA accumulator is implemented in VHDL as shown in Listing 1.

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  architecture rtl of da_accumulator is
6      signal acc : signed(23 downto 0);
7      signal lut_sum : signed(15 downto 0);
8  begin
9      process(clk, rst)
10     begin
11         if rst = '1' then
12             acc <= (others => '0');
13         elsif rising_edge(clk) then
14             if bit_idx < B - 1 then
15                 acc <= shift_right(acc, 1) +
16                     resize(lut_sum, 24);
17             else -- MSB (sign bit)
18                 acc <= shift_right(acc, 1) -
19                     resize(lut_sum, 24);
20             end if;
21         end if;
22     end process;
23 end architecture rtl;
```

Listing 1: DA accumulator core process

3.6 Algorithm Summary

The complete computation flow is formalised in Algorithm 1.

4 Results

4.1 Experimental Setup

The proposed architecture was implemented in VHDL and synthesised for a Xilinx Artix-7 XC7A100T-1CSG324C device using Vivado 2023.2. The filter specifications are summarised in Table 1.

Coefficients were designed using the Parks–McClellan algorithm (equiripple) and quantised to 16-bit fixed-point. The reference SNR of the floating-point implementation is $\text{SNR}_{\text{ref}} = 96.3$ dB.

Algorithm 1: Symmetric DA FIR Filter

Input: Input stream $x[n]$, coefficients $\{h_k\}_{k=0}^{31}$, word length B
Output: Filtered output $y[n]$
 // Pre-compute LUT contents offline
 1 **for** $m \leftarrow 0$ **to** $M - 1$ **do**
 2 **for** $\mathbf{a} \leftarrow 0$ **to** $2^P - 1$ **do**
 3 $\text{LUT}_m[\mathbf{a}] \leftarrow \sum_{j=0}^{P-1} a_j \cdot h_{mP+j}$;
 // Runtime: process each sample
 4 **for** each new sample $x[n]$ **do**
 5 Compute pre-added pairs:
 $s_k \leftarrow x[n - k] + x[n - 63 + k]$, $k = 0, \dots, 31$;
 6 $\text{acc} \leftarrow 0$;
 7 **for** $i \leftarrow 0$ **to** $B - 1$ **do**
 8 $T_i \leftarrow \sum_{m=0}^{M-1} \text{LUT}_m[s_{mP:mP+P-1}^{(i)}]$;
 9 **if** $i < B - 1$ **then**
 10 $\text{acc} \leftarrow \text{acc}/2 + T_i$;
 11 **else**
 12 $\text{acc} \leftarrow \text{acc}/2 - T_i$; // sign correction
 13 $y[n] \leftarrow \text{acc}$;

Table 1: Filter specifications.

Parameter	Value
Filter type	Low-pass, linear phase (Type I)
Number of taps (N)	64
Sampling frequency	250 MHz
Passband edge	20 MHz
Stopband edge	30 MHz
Passband ripple	≤ 0.1 dB
Stopband attenuation	≥ 60 dB
Input word length (B)	16 bit
Coefficient word length	16 bit

4.2 Resource Utilisation

Table 2 compares the post place-and-route resource utilisation of three implementations: conventional MAC, classical DA, and the proposed symmetric DA.

The proposed design eliminates all DSP48E1 slices while reducing LUT usage by 42 % compared to classical DA and Block Random Access Memory (BRAM) usage by 50 %.

Example 4.0 — LUT savings calculation

Classical DA with $N = 64$ and $P = 4$ partitions requires $2 \times 8 \times 16 = 256$ LUT entries. The proposed symmetric approach requires only $8 \times 16 = 128$ entries, a $1 - 128/256 = 50\%$ reduction in LUT table depth, translating to 42 % reduction in overall LUT primitives after synthesis.

Table 2: Resource utilisation comparison (post P&R).

Resource	MAC	Classical DA	Proposed DA
LUTs	1 284	3 841	2 228
Flip-Flops	2 048	1 536	1 312
DSP48E1	64	0	0
BRAM (36Kb)	0	4	2
$f_{\max}$ [MHz]	312	289	305

4.3 Power Analysis

Dynamic power was estimated using Vivado Power Analysis with switching activity from a 10 ms post-P&R simulation. Results are shown in Table 3.

Table 3: Power consumption comparison at 250 MHz.

Component	MAC	Classical DA	Proposed DA
Logic [mW]	18.2	14.7	11.3
Signal [mW]	22.5	16.8	13.1
DSP [mW]	31.4	0	0
BRAM [mW]	0	8.6	4.3
Clock [mW]	4.1	3.9	3.8
Dynamic [mW]	76.2	44.0	32.5
Static [mW]	45.3	45.3	45.3
Total [mW]	121.5	89.3	77.8

The proposed architecture achieves a dynamic power of 32.5 mW, representing a 57 % reduction compared to the MAC baseline and a 26 % reduction compared to classical DA.

4.4 SNR Performance

The SNR degradation due to fixed-point quantisation was measured by comparing the filter output against the ideal floating-point response across 10^6 random input samples:

$$\text{SNR} = 10 \log_{10} \frac{\mathbb{E}[y_{\text{ideal}}^2[n]]}{\mathbb{E}[(y_{\text{ideal}}[n] - y_{\text{fixed}}[n])^2]} \quad (9)$$

The measured SNR is 96.0 dB for the proposed architecture, representing a degradation of only $\Delta \text{SNR} = 0.3$ dB relative to the floating-point reference — well within acceptable limits for SDR channelisation applications where typical ADC SNR is below 70 dB.

Remark. The negligible SNR degradation confirms that the coefficient symmetry exploitation introduces no additional quantisation noise, as the pre-addition is performed on full-precision input samples before bit-slicing.

4.5 Comparison with State of the Art

Table 4 summarises the comparison with recent FPGA-based FIR implementations from the literature.

Table 4: Comparison with state-of-the-art FPGA FIR implementations.

	[2]	[4]	Classical DA	This work
Taps	32	16	64	64
f_s [MHz]	200	100	289	305
LUTs	890	412	3841	2228
DSP slices	32	16	0	0
Power [mW]	52	28	89	48*

*Total power at 250 MHz operating frequency.

5 Conclusion

This paper presented an energy-efficient FIR filter architecture for FPGA-based SDR receivers. By combining coefficient symmetry exploitation with a partitioned DA scheme and pipelined offset-binary coding, the proposed design achieves significant improvements over conventional implementations:

- **42 % LUT reduction** compared to classical DA, by halving the address space through pre-addition of symmetric input pairs.
- **37 % dynamic power reduction** compared to MAC-based implementations, by eliminating all DSP48E1 multiplier slices.
- $\Delta \text{SNR} < 0.3$ dB, confirming that the architectural optimisations introduce negligible quantisation degradation.
- $f_{\max} = 305$ MHz, exceeding the 250 MHz target with 22 % timing margin.

The architecture was validated on a Xilinx Artix-7 XC7A100T device with a 64-tap linear-phase low-pass filter for a 20 MHz-bandwidth SDR channelisation scenario.

5.1 Future Work

Several directions for future work are envisaged:

1. **Adaptive coefficients:** extending the architecture to support runtime coefficient updates for adaptive filtering applications, which would require a LUT refresh mechanism operating in $\mathcal{O}(2^P)$ cycles.
2. **Multi-rate support:** implementing polyphase decomposition to support decimation/interpolation directly within the DA structure.
3. **Comparison with HLS:** evaluating whether high-level synthesis tools can achieve comparable power-performance trade-offs automatically.
4. **ASIC migration:** porting the design to a 28 nm ASIC process to quantify power savings at lower technology nodes.

Note: The VHDL source code and testbench for the proposed architecture are available upon request from the corresponding author.

References

- [1] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th. Pearson Prentice Hall, 2007.
- [2] J. Smith and J. Doe, “Low-power FIR filter implementation on FPGA using distributed arithmetic,” in *Proceedings of the IEEE International Symposium on Circuits and Systems (ISCAS)*, 2023, pp. 1–5. DOI: [10.1109/ISCAS.2023.XXXXXXX](https://doi.org/10.1109/ISCAS.2023.XXXXXXX)
- [3] *IEEE standard VHDL language reference manual*, IEEE Std 1076-2008, Revision of IEEE Std 1076-2002, 2009.
- [4] B. Razavi, “Design of analog cmos integrated circuits,” *IEEE Journal of Solid-State Circuits*, vol. 36, no. 7, pp. 1050–1063, 2001. DOI: [10.1109/JSSC.2001.XXXXXXX](https://doi.org/10.1109/JSSC.2001.XXXXXXX)